It is not yet known if this is the lowest bound for the *Union-Find* data structure. It has been proven[13], though, that `O(m * InvAck(m, n))` is a strict lower bound, where `InvAck(m, n)` is the true inverse Ackermann function.

I know, this is a lot to take in. But do not despair: turns out that we only need a few small changes to implement the *path compression* heuristic.

5.6.2 Implementing Balancing and Path Compression

We will now discuss the final implementation of our disjoint-set structure, including both the "tree balancing by rank" and "path compression" heuristics.

For each element, we'll have to store some information about its subtree. Therefore, we'll use a helper (private) class to gather all the info together.

Listing 5.8 Class Info

```
class Info
  function Info
(elem)
#1
    throw-if elem ==
null                                                    #2
    this.root =
elem;
#3
    this.rank =
1;
#4
```